

机器学习实验报告/大作业1

学号	姓名
22336221	汪宣彤

一、实验要求

使用支持向量机（SVM）和不同损失函数的线性分类器

【问题描述】

手动实现线性分类模型，分别使用Hinge Loss和Cross-Entropy Loss进行比较

除此之外，使用多项式实现一个分类器，画出拟合曲线，并且比较不同参数量下的结果

此外，训练一个采用不同核函数的SVM二分类器

请根据以下要求完成实验并提交报告和代码

【数据集】

UCI的乳腺癌数据集 (Breast Cancer Wisconsin)

数据集地址: [UCI Breast Cancer Dataset](#)

【任务要求】

1. 实现一个线性分类器，使用两种不同的损失函数：

- Hinge Loss （对应SVM的损失函数）
- Cross-Entropy Loss （对应于逻辑回归的损失函数）

比较两种损失函数在分类任务中的表现

2. 使用多项式回归实现一个分类器，并比较不同参数量下的结果，从中体会什么是过拟合

3. 支持向量机SVM：选择两种核函数来训练SVM模型：

- 线性核函数
- 高斯核函数（RBF）

2和3可以使用现成的SVM软件包，例如Python的 scikit-learn 库

【实验报告】（内容包括但不限于以下部分）

1. 比较两种损失函数（Hinge Loss 和 Cross-Entropy Loss）在线性分类模型中的表现
2. 比较多项式分类器不同参数情况下的情况，可视化拟合曲线
3. SVM模型的一般理论：简要介绍SVM的基本原理，包括支持向量、间隔最大化、核函数等内容
4. 不同核函数的SVM模型与性能比较：比较线性核函数和高斯核函数在给定数据集上的分类性能，并分析结果

5. SVM的训练过程：详细描述训练过程，包括：

- 模型初始化方法
- 超参数选择（例如惩罚参数C、核函数参数等）
- 使用的训练技巧（例如正则化、梯度下降等）

6. 实验结果分析与讨论：包括模型的准确率、精确率、召回率、F1值等评价指标，并进行分析和讨论

二、实验1：线性分类器与两种损失函数比较

实验原理

1. 线性分类器

线性分类器的基本假设是数据是**线性可分**的。在二分类问题中，线性分类器的模型可以表示为：

$$y = \mathbf{w}^T \mathbf{x}$$

其中：

- $\mathbf{w}$ 是模型的权重向量
- $\mathbf{x}$ 是输入样本的特征向量

模型通过调整权重 $\mathbf{w}$ 来学习如何对数据进行分类。不同的损失函数用来度量模型输出与真实标签之间的差距，并通过梯度下降法来优化该差距

2. 损失函数

损失函数是衡量模型预测与真实标签之间误差的函数。它决定了如何惩罚不准确的预测，从而指导模型的训练过程

◦ Hinge Loss

Hinge Loss 通常用于 **支持向量机 (SVM)**。它的目标是最大化分类的间隔，使得数据点不仅被正确分类，还要离决策边界尽可能远。其定义如下：

$$L(y, f(\mathbf{x})) = \max(0, 1 - yf(\mathbf{x}))$$

其中：

- y 是真实标签
- $f(\mathbf{x})$ 是模型的预测

Hinge Loss 强制要求正确分类的点离决策边界至少距离 1，从而有助于间隔最大化

◦ Cross-Entropy Loss

Cross-Entropy Loss 通常用于 **逻辑回归**，广泛应用于二分类任务。其定义如下：

$$L(y, p(\mathbf{x})) = -[y \log(p(\mathbf{x})) + (1 - y) \log(1 - p(\mathbf{x}))]$$

其中：

- $p(\mathbf{x})$ 是模型预测的概率

交叉熵衡量了预测概率分布与真实标签之间的差异，越接近真实值损失越小

3. 性能评估指标

本次实验中，通过以下几个指标来评估模型的分类性能

- **准确率 (Accuracy)**
模型预测正确的样本数占总样本数的比例
- **精确率 (Precision)**
在所有预测为正类的样本中，实际为正类的样本所占的比例
- **召回率 (Recall)**
在所有实际为正类的样本中，被正确预测为正类的样本所占的比例
- **F1 分数 (F1 Score)**
精确率和召回率的调和平均数，综合衡量分类器的性能

关键代码展示

```
# 线性分类器
class LinearClassifier:
    def __init__(self, loss='hinge', learning_rate=0.01, n_epochs=1000):
        self.loss = loss
        self.learning_rate = learning_rate
        self.n_epochs = n_epochs
        self.weights = None

    def fit(self, x, y):
        num_samples, num_features = x.shape # 获取样本数和特征数
        # 初始化权重为零
        self.weights = np.zeros(num_features)

        for epoch in range(self.n_epochs):
            for i in range(num_samples):
                xi = x[i] # 当前样本的特征
                yi = y[i] # 当前样本的标签
                # 计算预测值
                prediction = np.dot(xi, self.weights) # 线性组合

                if self.loss == 'hinge':
                    # Hinge Loss的实现
                    # 如果预测的结果不满足条件，则更新权重
                    if yi * prediction < 1:
                        self.weights += self.learning_rate * (yi * xi - 2 *
self.weights)
                else:
                    self.weights -= self.learning_rate * 2 * self.weights

            elif self.loss == 'cross_entropy':
                # Cross-Entropy Loss的实现
                prob = 1 / (1 + np.exp(-prediction)) # Sigmoid函数得到概率
                gradient = prob - yi # 计算梯度
                self.weights -= self.learning_rate * gradient * xi # 更新权重

    def predict(self, x):
```

```
        predictions = np.dot(X, self.weights)
        return (predictions > 0).astype(int) # 将预测结果转换为0或1

# 评估模型性能
def evaluate_model(y_true, y_pred):
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    return accuracy, precision, recall, f1
```

代码输出

```
Hinge Loss Metrics:
Accuracy: 0.4234, Precision: 0.4234, Recall: 1.0000, F1 Score: 0.5949

Cross-Entropy Loss Metrics:
Accuracy: 0.8175, Precision: 0.7324, Recall: 0.8966, F1 Score: 0.8062
```

结果分析

1. 准确率 (Accuracy)

- Hinge Loss 的准确率为 42.34%，即只有大约 42.34% 的测试样本被正确分类
- Cross-Entropy Loss 的准确率显著提高，达到 81.75%，表明使用交叉熵损失函数的模型能够更有效地识别类别

2. 精确率 (Precision)

- Hinge Loss 的精确率为 42.34%，表示在所有被分类为正类的样本中，只有 42.34% 实际上是正类
- Cross-Entropy Loss 的精确率提升至 73.24%，意味着模型在正类预测中更为准确

3. 召回率 (Recall)

- Hinge Loss 的召回率为 100%，表示所有实际的正类样本都被正确识别出来
- Cross-Entropy Loss 的召回率为 89.66%，这表明模型在识别正类样本时存在一定的漏报，但仍然保持较高的识别能力

4. F1 Score

- Hinge Loss 的F1 Score为 0.5949，显示了较低的平衡性能，尤其是在精确率和召回率之间的权衡方面
- Cross-Entropy Loss 的F1 Score为 0.8062，表示该模型在精确率和召回率之间达成了更好的平衡，表现出色

讨论

- **Hinge Loss 的表现**

Hinge Loss 通常用于支持向量机 (SVM)，在该实验中，尽管其召回率很高，但准确率和精确率较低，表明模型在正类和负类样本分类上存在一定偏差

- **Cross-Entropy Loss 的表现**

Cross-Entropy Loss 使用交叉熵损失函数的模型显著提高了分类性能，尤其是在准确率和精确率方面，表明它能够更好地处理样本间的复杂关系

- **模型选择**

本实验中，Cross-Entropy Loss 的结果优于 Hinge Loss，建议在实际应用中优先考虑使用交叉熵损失进行二分类任务，特别是在特征数量较多或样本不均衡的情况下

三、实验2：多项式分类器，不同多项式阶数下的表现及过拟合现象的观察

实验原理

多项式回归

多项式回归是对线性回归的一种扩展，用来拟合非线性关系的数据。通过将输入特征 x 转换为多项式形式，即 x^d 的形式，多项式回归可以拟合更加复杂的曲线。因此，通过调整多项式的次数 d ，可以控制模型的复杂度。过高的次数往往会导致过拟合，表现为模型对训练数据拟合得过于精准，缺乏泛化能力

关键代码展示

```
# 定义一个函数来生成多项式特征并训练模型
def polynomial_regression(degree):
    # 多项式特征生成
    poly = PolynomialFeatures(degree=degree)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    # 使用逻辑回归作为分类器
    model = LogisticRegression(max_iter=10000)
    model.fit(X_train_poly, y_train)

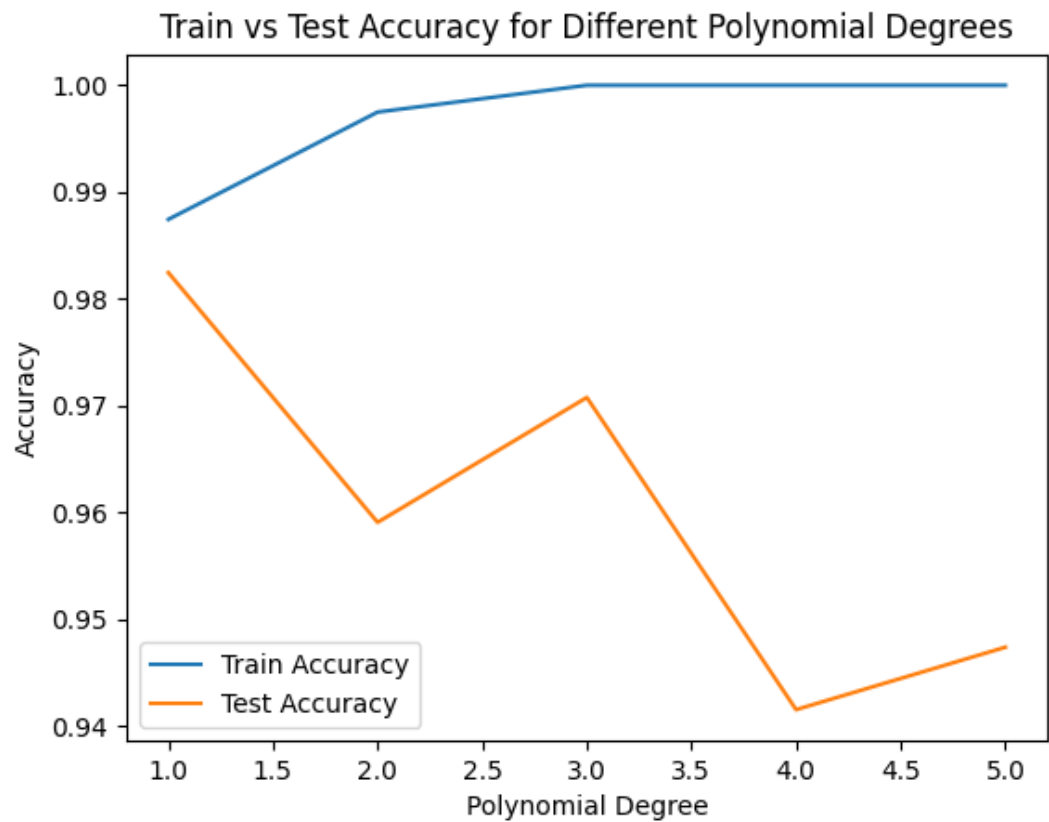
    # 预测并计算性能指标
    y_pred_train = model.predict(X_train_poly)
    y_pred_test = model.predict(X_test_poly)

    train_acc = accuracy_score(y_train, y_pred_train)
    test_acc = accuracy_score(y_test, y_pred_test)
    precision = precision_score(y_test, y_pred_test)
    recall = recall_score(y_test, y_pred_test)
    f1 = f1_score(y_test, y_pred_test)

    return train_acc, test_acc, precision, recall, f1
```

代码输出

```
Degree: 1
Train Accuracy: 0.9874371859296482, Test Accuracy: 0.9824561403508771
Precision: 0.9906542056074766, Recall: 0.9814814814814815, F1 Score: 0.986046511627907
-----
Degree: 2
Train Accuracy: 0.9974874371859297, Test Accuracy: 0.9590643274853801
Precision: 0.963302752293578, Recall: 0.9722222222222222, F1 Score: 0.967741935483871
-----
Degree: 3
Train Accuracy: 1.0, Test Accuracy: 0.9707602339181286
Precision: 0.9724770642201835, Recall: 0.9814814814814815, F1 Score: 0.9769585253456221
-----
Degree: 4
Train Accuracy: 1.0, Test Accuracy: 0.9415204678362573
Precision: 0.9454545454545454, Recall: 0.9629629629629629, F1 Score: 0.9541284403669725
-----
Degree: 5
Train Accuracy: 1.0, Test Accuracy: 0.9473684210526315
Precision: 0.9626168224299065, Recall: 0.9537037037037037, F1 Score: 0.958139534883721
-----
```



结果分析

1. 训练集准确率 (Train Accuracy)

随着多项式次数增加，训练集的准确率逐渐提升，并且在多项式次数 $d = 3$ 之后趋近于 1.0，表明模型在训练集上拟合得非常好

2. 测试集准确率 (Test Accuracy)

- 测试集准确率先是从 $d = 1$ 到 $d = 2$ **逐渐下降**，然后在 $d = 3$ 稍微提升，之后再度下降，在 $d = 4$ 达到了一个**最低点**
- 这表明模型在多项式次数增加到一定程度时，在测试集上的泛化性能下降，即出现了**过拟合**

3. 过拟合现象

- 过拟合** 是指模型在训练集上表现得非常好，但在测试集上的性能较差。随着多项式次数增加，模型对训练数据的拟合能力增强，但它也变得更加复杂，容易过拟合训练集中的噪声
- 在图中，**训练集的准确率几乎达到100%**，而**测试集准确率下降**，特别是 $d = 4$ 和 $d = 5$ 时，测试集准确率远低于训练集的准确率，表明模型在这些次数下已经过拟合

4. 最佳多项式次数

在图中， $d = 2$ 或 $d = 3$ 时，测试集的准确率较高，表明此时模型的泛化能力较好，没有显著的过拟合或欠拟合现象

结论

- 多项式次数越高，模型复杂度越大**，可能会更好地拟合训练集，但它可能会导致测试集性能下降，即出现过拟合
- 在本次实验中，**多项式次数 $d = 2$ 或 $d = 3$** 是比较合适的，既能保持较高的测试集准确率，也避免了过度拟合

四、使用线性核函数和高斯核函数的SVM

实验原理

1. 基本概念

SVM是一种监督学习模型，主要用于二分类任务。它通过在高维空间中寻找一个**最优的超平面**将数据集的**不同类别进行分离**。SVM的目标是找到间隔最大的决策边界（即超平面），并且**最大化类间的距离**，从而增强模型的泛化能力

2. 超平面和分类

在二维平面上，超平面就是一条直线，在三维空间上是一个平面。而在高维空间中，超平面是维度减1的一个几何对象。对于一个给定的数据集，SVM通过公式构造超平面： $\mathbf{w} \cdot \mathbf{x} + b = 0$ 其中， $\mathbf{w}$ 是超平面的法向量，决定了超平面的方向； b 是偏置项，控制超平面的位置

3. 间隔最大化

SVM的核心思想是**间隔最大化**

对于给定的超平面，**距离超平面最近的样本点称为支持向量**，这些点**决定了超平面的最终位置**。它们在训练模型时起关键作用，而其他远离超平面的点则对超平面的确定没有影响

SVM在训练过程中不仅寻找能正确分类数据的超平面，还要求这个超平面与支持向量之间的距离（间隔）最大化，从而使模型对数据有更好的泛化能力

4. 软间隔和惩罚参数C

在真实问题中，数据通常是线性不可分的，SVM通过引入**软间隔**来处理这种情况。软间隔允许一些数据点可以位于错误的一侧，但会对错误分类施加惩罚。惩罚程度由参数**C**决定：

- 大的C值会强制模型尽可能正确分类所有训练样本，但可能导致过拟合
- 小的C值则允许更多的分类错误，但有更好的泛化能力

5. 核函数

核函数本质上是一个**相似度函数**，它接收两个输入样本的特征向量 $\mathbf{x}$ 和 $\mathbf{x}'$ ，并输出一个表示它们相似度的数值。SVM不仅可以在线性可分的数据上使用，还可以通过**核技巧**扩展到非线性问题。核函数通过将低维数据映射到高维空间，使得在高维空间中可以找到线性可分的超平面。

本次实验使用的核函数如下：

- **线性核**
适用于线性可分的数据，公式为 $\mathbf{k}(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$
- **高斯核 (RBF核)**
常用于非线性可分的数据，公式为 $\mathbf{k}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ ， γ 控制高斯分布的宽度

6. SVM的优化目标

SVM的目标是通过最小化以下损失函数来实现间隔最大化：

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b))$$

其中，前半部分是用来最大化间隔的，后半部分是对错误分类点的惩罚

7. 线性核和高斯核 (RBF核) 对比

- **线性核函数**
适用于数据线性可分的情况。它简单且计算效率高，但在数据线性不可分的情况下性能较差
- **高斯核 (RBF核)**
是非线性问题中常用的核函数。它通过将数据映射到高维空间，使得即使在低维空间中不可分的数据，也可能在高维空间中可分

关键代码展示

1. 模型的初始化方法

在代码中，我们使用了 `svc` 类来初始化SVM模型

- **线性核 SVM**

```
linear_svm = SVC(kernel='linear', C=C_linear, random_state=42)
```

- `kernel='linear'`：指定使用线性核函数
- `C=C_linear`：将惩罚参数C设置为1.0，以控制分类的复杂性

- 高斯核 SVM (RBF)

```
rbf_svm = SVC(kernel='rbf', C=C_rbf, gamma=gamma_rbf, random_state=42)
```

- `kernel='rbf'`: 指定使用高斯核 (RBF) 函数
- `C=C_rbf`: 同样将惩罚参数C设置为1.0
- `gamma=gamma_rbf`: 设置为 `'scale'`, 表示根据特征的标准差自动选择 γ 值

2. 超参数的选择

- 惩罚参数C
 - 控制模型对分类错误的容忍度
 - 较小的C值会导致更强的正则化, 可能导致欠拟合; 而较大的C值则可能导致模型过拟合
- 高斯核的 γ 参数
 - 控制高斯核的宽度, 影响决策边界的复杂性
 - 较小的 γ 会导致较平滑的决策边界, 可能导致欠拟合; 较大的 γ 会导致更复杂的决策边界, 可能导致过拟合
 - 在代码中使用 `'scale'`, 可以自动根据数据的特征标准差来调整 γ

3. 使用的训练技巧

在训练SVM模型时, 采用以下训练技巧

- 数据标准化

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # 拟合并转换训练集
X_test = scaler.transform(X_test)      # 使用相同的标准化器转换测试集
```

- 通过 `StandardScaler` 对特征值进行标准化, 以确保每个特征的均值为0, 标准差为1

- 训练过程

在每个模型的初始化后, 使用 `.fit()` 方法对模型进行训练

```
linear_svm.fit(X_train, y_train)
rbf_svm.fit(X_train, y_train)
```

在这个过程中, 模型会根据训练集中的特征和目标值调整其参数, 以最大化间隔并最小化分类错误

代码输出

```
Linear Kernel SVM Metrics:
Accuracy: 0.9561, Precision: 0.9318, Recall: 0.9535, F1 Score: 0.9425

RBF Kernel SVM Metrics:
Accuracy: 0.9825, Precision: 1.0000, Recall: 0.9535, F1 Score: 0.9762
```

结果分析

1. 准确率 (Accuracy)

高斯核 SVM 的准确率 (0.9825) 显著高于线性核 SVM 的准确率 (0.9561), 这表明高斯核模型在整个数据集上表现更好

2. 精确率 (Precision)

- 高斯核 SVM 的精确率为 1.0000, 表示所有被预测为正类 (恶性肿瘤) 的样本都是正确的
- 线性核 SVM 的精确率为 0.9318, 表明部分负类样本被错误分类为正类

这种差异说明高斯核模型在识别恶性肿瘤方面表现得更加准确

3. 召回率 (Recall)

两个模型的召回率都是 0.9535, 说明它们在识别恶性肿瘤的能力上相似, 能够识别出绝大多数的恶性肿瘤样本

4. F1 Score

高斯核 SVM 的 F1 分数 (0.9762) 高于线性核 SVM 的 F1 分数 (0.9425)。F1 分数是精确率和召回率的调和平均, 表明高斯核 SVM 在平衡这两个指标方面表现更好

结论

- 从这些指标来看, 高斯核 SVM 模型在所有评价指标上均表现优于线性核 SVM, 尤其是在精确率方面
- 线性核 SVM 的性能虽然也不错, 但在处理非线性可分的数据时, 高斯核 SVM 更能发挥其优势